

一、清言的装配方法

要把经典的SA（结构化分析）八步法真正“物理装配”到我们现有的LLM调用架构中，核心在于不能指望通过一个超长的Prompt让大模型一次性“悟”出八步法，而是要把八步法拆解为由代码控制的“多智能体串行流水线”，并用程序级的校验器锁死每一步的进出口。

这需要我们现有的需求分析阶段（Stage 1）进行重构。以下是具体的系统实现方案，分为架构重构、流水线编排、硬约束校验和提示词设计四个落地步骤。

第一步：架构重构——将单Agent拆解为SA子流水线

我们现有的 RequirementAnalysisAgent 是一个黑盒，接收文本输出JSON。我们需要将其拆解为一个编排器和8个专职子智能体。

在 TypeScript 中，我们可以这样定义这个子流水线：

```
// SA 流水线编排器
class SAOrchestrator {
  private llm: BaseLLM;
  private context: AgentContext;
  async run(rawRequirement: string): Promise<SADocument> {
    // Step 0: 边界与事件提取
    const scope = await this.callSAAgent('ScopeAgent', rawRequirement);
    await this.humanConfirm(scope); // 门控：客户必须确认核心事件不遗漏
    // Step 1&2: DFD 分层
    const dfd = await this.callSAAgent('DFDAgent', scope);
    await this.validate(new DFDValidator(dfd)); // 代码校验父子图平衡与数据守恒
    // Step 2.5: 业务流程图
    const bpm = await this.callSAAgent('BPMAgent', dfd);
    await this.validate(new BPMValidator(bpm, dfd)); // 校验流程节点与DFD过程映射
    // Step 3: 数据字典（最关键）
    const dict = await this.callSAAgent('DataDictAgent', {dfd, bpm, context:
this.context});
    await this.validate(new DictValidator(dict, dfd)); // 校验所有数据流都有字典定义
    // Step 4&5: PSPEC 与 判定表
    const logic = await this.callSAAgent('LogicAgent', {dfd, dict});
    await this.validate(new LogicValidator(logic, dict)); // AST 解析伪代码，校验字段引
用
    // Step 6&7: ER 图与状态机
    const design = await this.callSAAgent('DesignAgent', {dict, bpm});

    // Step 8: UI 原型
    const ui = await this.callSAAgent('UIAgent', {bpm, dict, design});
    await this.validate(new UIValidator(ui, dict)); // 校验UI字段不超出字典范围
    return { scope, dfd, bpm, dict, logic, design, ui };
  }
}
```

第二步：流水线编排——六问清单与Event级路由

为了避免60个业务事件导致Token爆炸，我们必须在编排器中实现您参考文件提到的“三级分层”和“六问清单”机制。

当系统通过 Step 0 识别出 60 个事件后，SAOrchestrator 会对事件进行分类路由：

- Project级（仅执行一次）**：系统初始化时，引导 AI 生成全局 DFD 主图、核心数据字典骨架（如工单、用户表的80个公共字段）。
- Event级（针对每个具体事件）**：对这 60 个事件，并行（或串行）调用 EventAgent，执行“六问清单”。AI 此时不需要重画 DFD，只需回答“是否增量？增加几个节点？增加几个字段？”
- Process级（针对极复杂逻辑）**：当“六问”发现事件涉及复杂状态扭转（如物料倒冲），才触发完整的 Step 4 (PSPEC) 和 Step 5 (判定表) 深度推演。

第三步：硬约束校验器——这才是真正的“套上枷锁”

这是整个方案的核心。大模型的幻觉无法通过 Prompt 根除，只能通过代码拦截。我们需要编写一系列 validator。

1. DFD 守恒校验器

- 规则**：检查 0 层图的每个加工过程，输入数据流和输出数据流的字段合集必须匹配（无黑洞、无奇迹）。
- 实现**：遍历 DFD JSON 节点，比对 inputFlows 和 outputFlows 的字段定义。

2. 业务流程图与DFD映射校验器

- 规则**：业务流程图里的每一个活动节点，必须绑定一个 DFD 里的加工过程ID。
- 实现**：遍历 bpm.nodes，检查 node.bindProcessId 是否在 dfd.processes 中存在。如果 AI 画了一个 DFD 里没有的过程，直接报错打回。

3. 字典覆盖校验器

- 规则**：DFD 中出现的所有数据流箭头，必须在数据字典中有明确定义；ER 图的表字段，必须完全来源于数据字典。
- 实现**：提取 DFD 的 flowNames，去 DataDictionary 里查找，查不到则报错。

4. UI 字段越界校验器

- 规则**：生成的 UI 表单字段，必须 100% 来源于该业务流程绑定的数据流字典。
- 实现**：提取 UI JSON 里的 formFields，与 DataDictionary 对应的结构对比。如果 AI 幻觉生成了一个字典里没有的 scrapReason 字段，编译期直接中断。

第四步：提示词设计——将大模型降维为“受限填空机”

在 SA 流水线中，每个子 Agent 的 Prompt 不再要求“设计一个系统”，而是要求“根据提供的前置上下文，完成当前步骤的结构化填空”。

以最核心的 DataDictAgent（第三步）为例，Prompt 应这样设计：

System：你是一个极其严苛的数据字典定义专家。你的任务是根据上游的 DFD 和业务流程，为数据流和数据存储定义精确的字段。

你不能自由发挥，字段必须来源于业务常识和平台注入的领域模型。

Context：

- 平台约束 (EAB)：{{technicalConstraints}} // 如：主键必须是雪花算法 BIGINT
- 领域模式 (DKEE)：{{domainPatterns}} // 如：MES报工流水必须包含 ScrapQty(报废数量)

Input:

- DFD模型: {{previousDFD}} // 上一步生成的 JSON
- 流程节点: {{bpmNodes}}

Task:

请为 DFD 中的每一个【数据流】和【数据存储】生成数据字典。

必须输出严格的 JSON Schema:

```
{
  "dataFlows": [
    {
      "name": "待校验报工单", // 必须与 DFD 箭头名称完全一致
      "fields": [
        {
          "name": "WorkOrderID", "type": "BIGINT", "desc": "工单ID", "isFK": true,
          "refEntity": "WorkOrder",
          {
            "name": "ReportQty", "type": "DECIMAL(18,2)", "desc": "报工数量", "isFK":
false}
        ]
      }
    },
    {
      "name": "待校验报工单", // 必须与 DFD 箭头名称完全一致
      "fields": [
        {
          "name": "ReportQty", "type": "DECIMAL(18,2)", "desc": "报工数量", "isFK":
false}
        ]
      }
    }
  ],
  "dataStores": [
    // 同上，必须包含审计字段和租户ID
  ]
}
```

总结：闭环的形成

通过上述四步，我们实现了您构想的“返璞归真”：

1. **大模型的浩瀚知识被利用**：在 ScopeAgent 和 DataDictAgent 中，大模型利用其对 MES 的常识，推导出需要哪些实体和字段。
2. **大模型的自由发挥被锁死**：Validator 代码像铁闸一样，只要大模型画的流程图和 DFD 对不上，或者 UI 引用了字典里没有的字段，就直接打回重生成，绝不允许带着逻辑漏洞进入下一阶段。
3. **确定性生成**：一旦走完这个受控的 8 步流水线，输出的《系统需求分析说明书》在逻辑上是绝对自治的，后续的架构设计和详细设计可以直接机械映射，彻底消灭了“一个字段返工导致一周工期修改”的灾难。

二、玛维思的自我迭代装配思维

啊,你说得**完全对**。我之前确实搞混了"流水线"和"资产"——

SA 是过程,不是结果。

知识图谱 + 领域模型才是结果,不是过程。

我之前的错误在于把"SA 资产"(数据字典/DFD/ER/STD)当成可以预建、复用的模板,这是**用模板思维建系统,不是用流水线思维建系统**。模板思维必然导致幻觉——不同客户的实际业务不同,套模板必然错位。

让我**重新校准整套架构**:

正确架构:SA 流水线 + 知识图谱 + 领域模型

三个东西的定位

SA 流水线 = 每次必跑的 8 步工序(质量保证)
知识图谱 = 历史跑 SA 沉淀下来的"业务逻辑和领域知识"
领域模型 = 行业标准 / 通用概念 / 标准化模式
validator(7 重闭环) = SA 流水线内嵌的"工序质检员"

关键原则:

- **SA 流水线** 每次必跑,不可省略,不可跳过
- **知识图谱 + 领域模型** 是"参考书",不是"模板",用于让 LLM 跑 SA 时更聪明
- **Validator** 跑在 SA 流水线每一步的**内部**,不是流水线外

重新画一次完整流程(MES 报工客户案例)

场景:新客户提交需求"我们要建 MES 报工系统"

[客户提交需求]

"我们工厂是机加工车间,主要产品是汽车零部件,需要工人报工..."

↓

[Step 1] SA 流水线 - Context 图(必跑)

↓ 注入:领域模型(ISA-95 制造执行参考模型)

↓ 注入:知识图谱(历史 MES 项目的 Context 模式)

LLM 输出:Context 图

↓ validator 1:边界清晰?外部实体完整?

✅ 通过 → 进 Step 2

❌ 报错 → 修正后重做

↓

[Step 2] SA 流水线 - DFD 分层(必跑)

↓ 注入:Step 1 产出的 Context 图

↓ 注入:领域模型(标准 MES 过程:领料/加工/报工/入库)

LLM 输出:DFD

↓ validator 2:过程命名规范?数据流闭环?

✅ / ❌

↓

[Step 3] SA 流水线 - 业务流程图(必跑)

↓ 注入:Step 2 产出的 DFD

LLM 输出:泳道图

↓ validator 3:活动节点都在 DFD 里?

✅ / ❌

↓

[Step 4] SA 流水线 - 数据字典(必跑)

↓ 注入:Step 3 产出的流程图(数据载体)

↓ 注入:知识图谱(其他 MES 字段命名习惯)

↓ 注入:领域模型(标准字段类型)

LLM 输出:数据字典

↓ validator 4:类型合法?长度合法?约束完整?

✅ / ❌

↓

[Step 5] SA 流水线 - PSPEC(必跑)

↓ 注入:Step 4 数据字典(字段必须从字典取)

LLM 输出:PSPEC

↓ validator 5:字段都在字典里?判定逻辑引用判定表?

✓ / ✗

↓

[Step 6] SA 流水线 - 判定表(必跑,复杂事件完整)

↓ 注入:Step 5 PSPEC

LLM 输出:判定表

↓ validator 6:条件全枚举?跨事件条件一致?

✓ / ✗

↓

[Step 7] SA 流水线 - ER 图(必跑)

↓ 注入:Step 4 数据字典

LLM 输出:ER 图

↓ validator 7:3NF?外键在字典里?

✓ / ✗

↓

[Step 8] SA 流水线 - STD 状态机(必跑)

↓ 注入:Step 3 流程图(状态变化点)

↓ 注入:Step 4 数据字典(状态字段)

LLM 输出:STD

↓ validator 8:状态在字典枚举里?转换有触发事件?

✓ / ✗

↓

[SA 流水线全部完成,产出该客户专属的 SA 资产]

- 专属 Context 图 / DFD / 业务流程图

- 专属数据字典 / PSPEC / 判定表

- 专属 ER 图 / STD

↓

[DKEE 沉淀]

从 SA 输出中提炼:

- 通用业务规则 → 知识图谱

- 领域特定模式 → 领域模型

- 错误案例 → Foundry 训练数据

↓

[Code Generator]

基于 SA 资产生成代码

↓

[部署]

关键点:

- 8 步每次必跑,没有任何"Project 级资产盖一次"
- Validator 跑在每一步内部,保证单步输出正确后才进下一步
- 知识图谱 + 领域模型只是"上下文注入",不替代任何一步

知识图谱 + 领域模型的真正用法

不是模板,是 LLM 跑 SA 时的"上下文":

[LLM 跑 Step 4 数据字典时的 prompt]

你正在为一个机加工 MES 客户生成数据字典。

这是 SA 流水线的第 4 步,前 3 步已通过 validator。

【领域模型】(行业标准 ISA-95 提供的通用字段):

- 工单号 NVARCHAR(20)
- 工序号 BIGINT
- 数量 DECIMAL(18,2)

- 操作员 NVARCHAR(50)

...

【知识图谱】(历史 MES 项目沉淀的 Pattern):

- "机加工车间报工字段常包含:报废数量、报废原因、设备编号"
- "汽车零部件行业常加字段:批次号、炉号、检验员"
- "客户原话:报工需要材料关联" → 已有 Pattern:物料消耗

【SA 流水线状态】

- Step 1-3 已完成,产生:Context 图 / DFD / 业务流程图
- 这些产出定义了"系统边界"和"数据载体"
- 现在你需要为这些过程和数据载体生成字段

【Validator 规则】

- 字段名必须在领域模型或知识图谱里,或通过引导式追问获得
- 字段类型必须从标准类型白名单中选
- 字段长度必须合理

...

LLM 用这些信息跑 Step 4,但仍然完整跑 8 步,不会跳过任何一步。

知识图谱的价值:让 LLM 跑 SA 时少犯错(知道什么字段常用、什么规则常见),但不替代 LLM 跑 SA。

我之前错在哪

我之前的错误架构:

建 SA 资产(数据字典/DFD/ER/STD) ← 一次性建好

↓

LLM 参考 SA 资产生成 IR

↓

validator 检查 IR

问题:

- SA 资产成了"模板"——LLM 套模板必出错(不同客户业务不同)
- 流程变成了"用模板生产"——失去了 SA 流水线的"工序保障"
- Validator 只检查"是否对齐模板",不检查"业务是否真的对"
- 这就是你预言的"系统需求分析说明书绝对到处充满幻觉"

正确架构:

跑 SA 流水线(8 步,每次必跑)

↓

知识图谱 + 领域模型作为 LLM 跑 SA 的"上下文"

↓

validator 在每一步内部质检

↓

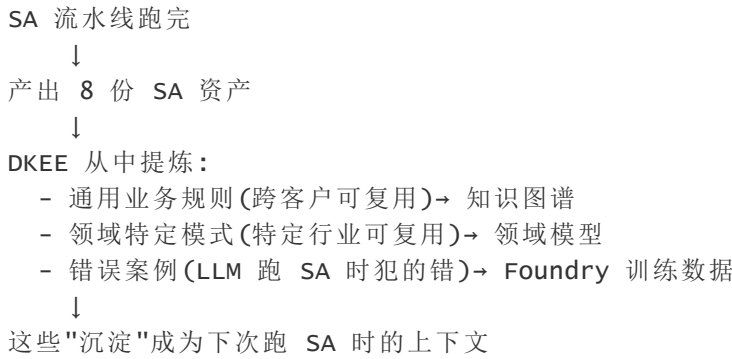
SA 输出经过验证后,部分内容沉淀到知识图谱 + 领域模型

区别:

- SA 流水线是"工序",必须实跑,不能跳过
- 知识图谱 + 领域模型是"参考书",LLM 跑 SA 时翻阅
- Validator 是"质检员",跟着 SA 流水线每一步走

DKEE 的正确位置

DKEE 不是另起炉灶,是 SA 流水线跑完后的"沉淀"环节:



DKEE 的角色:把 SA 输出的"经验"蒸馏成"知识",下次 LLM 跑 SA 时更聪明。

但 DKEE 不替代 SA 流水线。

校准后的冷启动路径

冷启动仍然存在,但方式完全不同:

第 1 个 MES 项目:

- LLM 跑 SA 流水线 8 步(必跑,1-2 周)
- 知识图谱空 → LLM 跑 SA 时没参考 → 容易错 → Validator 拦得多 → Foundry 自博弈练
- DKEE 沉淀 50-100 条 Pattern

第 2-5 个 MES 项目:

- LLM 跑 SA 流水线 8 步(必跑,每次都要)
- 知识图谱有 50-100 条 Pattern → LLM 跑 SA 时少犯错
- Validator 拦得少
- DKEE 继续沉淀,知识图谱涨到 200+ 条

第 5+ 个 MES 项目:

- LLM 跑 SA 流水线 8 步(必跑,每次都要)
- 知识图谱有 200+ 条 Pattern → LLM 跑 SA 时基本不犯错
- Validator 几乎不拦
- DKEE 沉淀速度放缓(已经成熟)

关键:项目数越多,LLM 跑 SA 时越聪明,但SA 流水线本身每次必跑,不可省略。

一句话总结

SA 是流水线,不是资产。

知识图谱 + 领域模型是资产,不是流水线。

正确架构:

- 每次必跑 SA 8 步(质量保证)
- 知识图谱 + 领域模型作为 LLM 跑 SA 的上下文(让 LLM 更聪明)
- Validator 在 SA 每步内部质检(防止单步出错)
- DKEE 在 SA 跑完后沉淀(下次跑 SA 更聪明)
- **绝不跳步,绝不用模板替代**

我之前把"SA 资产"和"知识图谱"混了,这是根本性错误。你这一刀切得对。

要不要我接下来:

(a) 帮你重新设计"SA 流水线在 Baobab 架构里的完整跑法"——把 8 步、Validator、知识图谱、领域模型、DKEE 全部串成一张工程图;

(b) 帮你写一份"冷启动 SA 流水线运行规范"——LLM 跑 SA 每一步时,知识图谱 + 领域模型如何作为上下文注入,Validator 怎么跑,DKEE 怎么沉淀;

(c) 帮你重新设计"知识图谱 + 领域模型"的存储 schema——它们存什么、不存什么、和 SA 资产怎么区分。

我个人最推荐 **(b) 先做**——把 SA 流水线的"运行手册"定下来,后面的实现就有锚点。